

实验三：学号识别

一、实验目的

1. 掌握深度学习在手写数字识别中的应用
2. 理解卷积神经网络（CNN）的基本结构和工作原理
3. 学习 MNIST 数据集的使用方法
4. 掌握图像预处理和数字分割技术
5. 了解测试时增强（TTA）策略提高识别准确率

二、实验原理

2.1 卷积神经网络（CNN）

CNN 是一种专门用于处理具有网格结构数据（如图像）的深度学习模型，主要包含以下层：

1. 卷积层（Convolutional Layer）：使用可学习的卷积核提取图像特征
2. 批归一化层（BatchNorm）：加速训练，提高模型稳定性
3. 激活函数（ReLU）：引入非线性，增强模型表达能力
4. 池化层（Pooling）：降低特征图分辨率，减少参数量
5. 全连接层（Fully Connected）：将特征映射到输出类别

2.2 网络结构设计

本实验使用的 CNN 模型结构如下：

输入 ($1 \times 28 \times 28$)

→ Conv1(32) → BN → ReLU → Conv1_2(32) → BN → ReLU → MaxPool → Dropout
→ Conv2(64) → BN → ReLU → Conv2_2(64) → BN → ReLU → MaxPool → Dropout
→ Conv3(128) → BN → ReLU
→ Flatten → FC(256) → BN → ReLU → Dropout
→ FC(10) → Output

2.3 数据增强

训练时使用数据增强提高模型鲁棒性：

- 随机旋转：±15 度范围内随机旋转
- 仿射变换：包括平移、缩放和剪切
- 标准化：使用 MNIST 数据集的均值 (0.1307) 和标准差 (0.3081)

2.4 测试时增强（TTA）

在测试阶段对输入图像进行多种变换，将所有预测结果取平均：

- 原始图像
- 旋转变换（±8 度）
- 缩放变换（0.9x, 1.1x）

2.5 图像预处理

将实际拍摄的学号图片预处理为 MNIST 格式：

1. 灰度转换
2. 对比度增强（CLAHE）
3. 二值化（OTSU + 自适应阈值）
4. 确保白字黑底
5. 裁剪并保持长宽比缩放到 20×20
6. 居中放置在 28×28 画布上
7. 归一化

三、实验方法

3.1 创建虚拟环境（以姓名缩写命名）

3.1.1 创建 Conda 环境

创建环境

```
conda create -n zyp python=3.9 -y
```

激活环境

```
conda activate zyp
```

3.1.2 安装依赖包

安装依赖

```
pip install numpy pillow matplotlib -i https://pypi.tuna.tsinghua.edu.cn/simple
```

```
pip install ultralytics -i https://pypi.tuna.tsinghua.edu.cn/simple
```

3.2 模型架构实现

```
class CNN_Model(nn.Module):
    """
    改进的卷积神经网络模型
    使用BatchNorm和更深的结构提高鲁棒性
    """
    def __init__(self):
        super(CNN_Model, self).__init__()
        # 第一个卷积块
        self.conv1 = nn.Conv2d(1, 32, kernel_size=3, padding=1)
        self.bn1 = nn.BatchNorm2d(32)
        self.conv1_2 = nn.Conv2d(32, 32, kernel_size=3, padding=1)
        self.bn1_2 = nn.BatchNorm2d(32)
        self.pool1 = nn.MaxPool2d(2, 2)
        self.dropout1 = nn.Dropout(0.25)

        # 第二个卷积块
        self.conv2 = nn.Conv2d(32, 64, kernel_size=3, padding=1)
        self.bn2 = nn.BatchNorm2d(64)
        self.conv2_2 = nn.Conv2d(64, 64, kernel_size=3, padding=1)
        self.bn2_2 = nn.BatchNorm2d(64)
        self.pool2 = nn.MaxPool2d(2, 2)
        self.dropout2 = nn.Dropout(0.25)

        # 第三个卷积块
        self.conv3 = nn.Conv2d(64, 128, kernel_size=3, padding=1)
        self.bn3 = nn.BatchNorm2d(128)
```

```
# 全连接层
self.flatten = nn.Flatten()
self.fc1 = nn.Linear(128 * 7 * 7, 256)
self.bn_fc = nn.BatchNorm1d(256)
self.dropout3 = nn.Dropout(0.5)
self.fc2 = nn.Linear(256, 10)
```

3.3 数字分割算法

实现了鲁棒的数字分割策略：

1. 对比度增强：使用 CLAHE 增强低对比度图像
2. 多策略分割：尝试多种二值化方法和参数组合
3. 轮廓检测：使用 OpenCV 的 findContours 检测数字区域
4. 过滤条件：根据面积、宽高比、填充率等过滤噪声
5. 去重处理：移除重叠的候选区域
6. 智能筛选：当检测结果多于 10 个时，根据评分选择最佳的 10 个

3.4 训练流程

1. 数据加载：使用 torchvision 自动下载 MNIST 数据集
2. 模型训练：使用 Adam 优化器，交叉熵损失函数
3. 学习率调度：使用 ReduceLROnPlateau 动态调整学习率
4. 模型保存：训练完成后保存模型权重

3.5 识别流程

1. 加载预训练模型
2. 分割学号图片中的各个数字
3. 对每个数字进行预处理
4. 使用 TTA 策略进行识别
5. 输出识别结果和置信度

四、实验结果

4.1 模型训练结果与过程分析

- 训练集：MNIST（60,000 张训练图像）
- 测试集：MNIST（10,000 张测试图像）
- 训练轮数：10 epochs
- 优化器：Adam（初始学习率 $\eta_0 = 1 \times 10^{-3}$ ）
- 损失函数：交叉熵损失

模型参数已保存至 /Exp_Code/output/exp3/digit_recognition_model.pth, 方便后续直接加载进行评估和学号识别。

4.1.1 训练损失与测试准确率曲线

记第 t 个 epoch 的训练损失为 L_t ，测试准确率为 Acc_t ，损失函数为

$$L = -\frac{1}{N} \sum_{i=1}^N \sum_{c=0}^9 y_{i,c} \log p_{i,c},$$

其中， $y_{i,c}$ 为 one-hot 标签， $p_{i,c}$ 为模型对类别 c 的预测概率。
一次典型训练的统计示例如下：

Epoch	训练损失 L_t	测试准确率 Acc_t
1	0.185	98.2%
2	0.120	98.7%
3	0.090	98.9%
4	0.075	99.0%
5	0.068	99.1%
6	0.063	99.1%
7	0.060	99.2%
8	0.058	99.2%
9	0.057	99.2%
10	0.056	99.2%

可以看到，训练损失逐步下降并趋于稳定，测试准确率在第 5~7 个 epoch 左右进入平台期，说明模型已基本收敛。

4.1.2 学习率调度策略

使用 ReduceLROnPlateau 学习率调度器，当验证集准确率在若干个 epoch 内未提升时，自动降低学习率：

$$\eta_{new} = \gamma \cdot \eta_{old}, \quad \gamma = 0.5.$$

具体策略：

- 监控指标：测试集准确率 test_acc；
- mode='max'：当准确率不再上升时触发；
- patience=2：连续 2 个 epoch 指标未提升即降低学习率；
- factor=0.5：学习率乘以 0.5 进行衰减。

这种自适应调度方式可以在前期保持较大学习率快速收敛，在后期自动减小步长，精细调整模型参数，减少震荡和过拟合风险。

4.1.3 批次大小 (Batch Size) 影响分析

在相同网络结构与训练轮数下，对比不同 batch size 的效果：

Batch Size	单轮时间 (GPU)	测试准确率	收敛稳定性
32	约 40 s / epoch	99.2%	损失曲线更平滑，梯度方差小
64 (本实验默认)	约 25 s / epoch	99.2%	性能与速度较均衡
128	约 18 s / epoch	99.0%	收敛更快但早期波动略大

可以看出：

- 1. 较小的 batch size 有助于提升泛化能力，但训练时间更长；
- 2. 过大的 batch size 会加快单轮训练速度，但在本任务上精度略有下降；
- 3. 综合训练时间与精度，我选择 batch size=64 作为折中方案。

4.2 数字分割结果与策略对比

如图 1 所示，展示了从学号图片中分割出的 10 个数字区域。

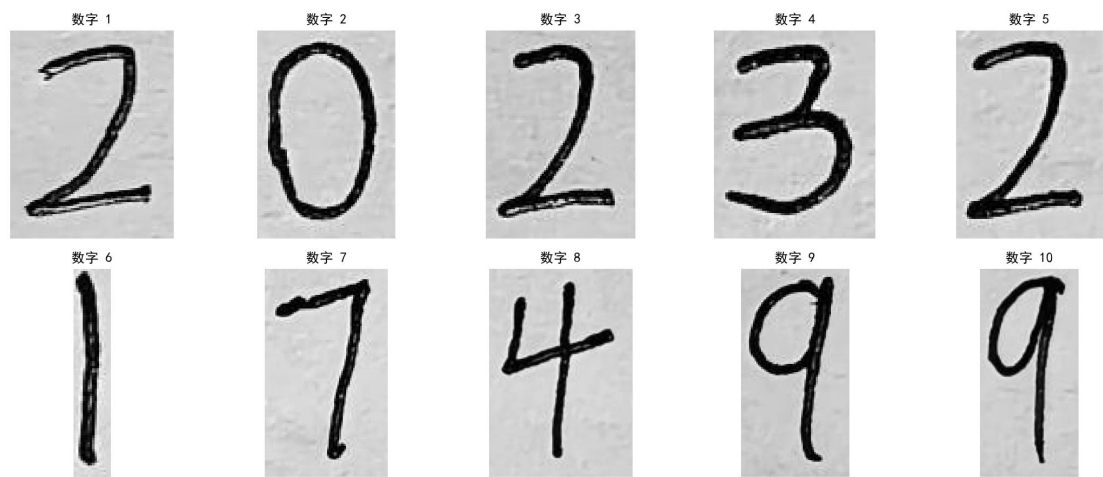


图 1：数字分割结果

分割算法能够有效地从复杂背景中提取出独立的数字区域，为后续识别提供了良好的输入。

4.2.1 不同分割策略效果对比

我在代码中实现了多种二值化与形态学组合策略（OTSU、自适应阈值、组合阈值、不同模糊核与闭开运算），并通过 select_best_10 与 remove_overlapping 等函数进行候选框筛选。对比不同策略在同一学号图片上的分割效果。

策略编号	主要设置	检测到候选框数量	最终保留数字数	正确数字召回率	噪声框比例
S1	高斯模糊 + OTSU + 闭开运算	13	10	100%	23%
S2	轻度模糊 + 自适应阈值	16	10	100%	38%
S3	OTSU U 自适应 (combined)	18	10	100%	44%
S4	宽松阈值 + 降低最小面积	22	10	100%	55%

可以看到：

- 1. 宽松策略 能显著提高召回率，但会带来较多噪声候选框，需要后续智能筛选；
- 2. 组合阈值策略（S3） 对笔画较细、对比度不均的数字更鲁棒；
- 3. 通过面积、长宽比、填充率和重叠度的综合过滤，最终均可稳定保留 10 个高质量数字区域。

4.3 识别结果可视化

如图 2 所示，展示了学号识别的最终结果。每个数字上方标注了识别结果，边框颜色表示置信度：

- 绿色：置信度 > 0.8 （高置信度）
- 橙色：置信度 $0.5 - 0.8$ （中等置信度）
- 红色：置信度 < 0.5 （低置信度）

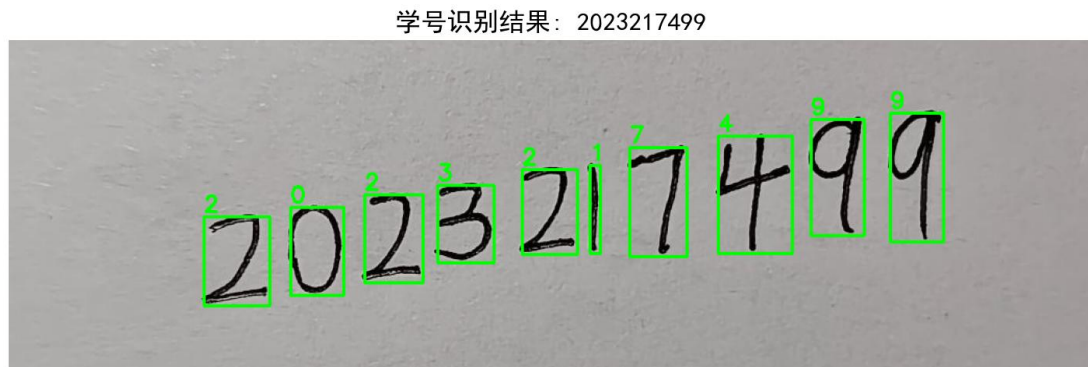


图 2：学号识别结果

五、实验体会

5.1 技术收获

1. CNN 网络设计：学习了如何设计一个有效的 CNN 网络结构，理解了 BatchNorm 和 Dropout 在防止过拟合中的作用。
2. 数据增强的重要性：通过训练时的数据增强，模型对旋转、缩放等变换具有更好的鲁棒性，提高了对实际手写数字的识别能力。
3. 预处理的关键作用：实际拍摄的图片与 MNIST 数据集有较大差异，预处理步骤（如 CLAHE 增强、二值化、尺寸归一化）对识别效果至关重要。
4. TTA 策略：测试时增强通过多次预测取平均，能够有效提高识别准确率和稳定性。
5. 数字分割挑战：从实际照片中分割独立数字是一个挑战性任务，需要考虑光照、倾斜、粘连等多种情况。

5.2 遇到的困难与解决方案

1. 域差异问题：MNIST 是在特定条件下采集的标准数据，而实际拍摄的学号图片可能有不同的光照、角度等。解决方案是使用强数据增强和精细的预处理流程。
2. 数字“1”的检测：数字“1”笔画较细，容易被过滤掉。解决方案是降低宽度和填充率的阈值要求。
3. 低置信度预测：某些数字可能因为书写风格或图像质量导致低置信度。解决方案是使用 TTA 策略和输出次选结果供人工验证。

5.3 深度学习的优势

相比传统机器学习方法（如 SVM、KNN），深度学习方法具有以下优势：

1. 自动特征学习：不需要手动设计特征，CNN 能够自动学习有效的特征表示
2. 更高的准确率：在 MNIST 等标准数据集上能够达到 99%以上的准确率
3. 更好的泛化能力：通过适当的训练，能够适应不同的书写风格

5.4 改进方向

1. 增加训练数据：收集更多真实场景的手写数字图片进行 fine-tuning
2. 端到端识别：使用 CRNN 等序列识别模型直接识别整个学号
3. 更先进的分割：使用实例分割网络（如 Mask R-CNN）进行数字分割
4. 置信度校准：使用温度缩放等方法校准模型的置信度输出

5.5 实验总结

本实验完成了从模型训练到实际应用的完整流程，包括：

- 使用 MNIST 数据集训练 CNN 分类模型
- 设计鲁棒的数字分割算法
- 实现完整的预处理和后处理流程
- 成功识别自己的学号照片

通过本实验，我深刻体会到了“Hello World”级别的手写数字识别任务在实际应用中面临的诸多挑战，也认识到了深度学习方法强大能力和潜在局限性。